

OneRec-V2 技术报告

中文译文

OneRec Team

原文标题: *OneRec-V2 Technical Report*

基于用户上传 PDF 整理

说明: 本译文按照原文段落顺序进行中文翻译, 尽量保留原有章节、公式编号、表格数据与图注含义。由于本次任务重点在于中文翻译与 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ /PDF 输出, 文中的原始图形未重新绘制, 改为保留对应图注的中文译文; 参考文献条目不逐条翻译, 使用时请结合原始论文查阅。

2026 年 4 月 2 日

目录

摘要	2
1 引言	2
2 惰性仅解码器架构	4
2.1 设计原则	4
2.2 整体架构	5
2.2.1 上下文处理器 (Context Processor)	6
2.2.2 惰性解码器块 (Lazy Decoder Block)	6
2.3 实证结果	7
2.3.1 架构比较	8
2.3.2 键值共享	8
2.3.3 分组查询注意力	9
2.3.4 模型扩展	9
3 基于真实用户交互的偏好对齐	11
3.1 基于用户反馈信号的强化学习	11
3.1.1 时长感知奖励塑形	11
3.1.2 强化学习	12
3.1.3 实验	14
3.2 用户反馈信号与奖励模型	15
3.2.1 奖励模型的局限	15
3.2.2 实验	16
4 在线 A/B 测试	17
5 结论、局限与未来方向	18
A 贡献说明	18
B 不同架构的计算复杂度	19
C 补充实证结果	20
D 关闭缓存时的在线性能	20

摘要

生成式人工智能的最新突破，通过实现端到端生成，从根本上改变了推荐系统。OneRec 作为一个工业级生成式推荐框架，将推荐重新表述为自回归生成任务，从而能够直接优化最终目标，并获得较高的模型 FLOPs 利用率（Model FLOPs Utilization, MFU）。尽管 OneRec-V1 已在真实部署中展现出显著的经验成功，但仍有两个关键问题制约其可扩展性与性能：其一，编码器-解码器架构中的计算分配低效，其中 97.66% 的资源消耗在序列上下文编码而非生成阶段，这限制了模型扩展；其二，仅依赖奖励模型的强化学习存在局限，包括采样效率较低，以及由于代理奖励信号而引发潜在的奖励黑客问题。为解决这些问题，我们提出 OneRec-V2，其核心包括：

1. **惰性仅解码器架构 (Lazy Decoder-Only Architecture)**：该架构采用精简的仅解码器设计，消除了编码器瓶颈并简化了交叉注意力，使总计算量降低 94%、训练资源降低 90%（见图 1 右）。这种效率使模型能够成功扩展到 8B 参数规模；更重要的是，其收敛损失与经验扩展律高度吻合。随着模型规模增大，我们观察到损失呈现平滑且可预测的下降趋势，这一趋势与扩展律拟合结果一致（见图 1 左与图 6）。
2. **基于真实用户交互的偏好对齐 (Preference Alignment with Real-World User Interactions)**：该框架以用户反馈为驱动，包含：(i) **时长感知奖励塑形**，用于缓解视频时长偏置；(ii) **自适应比率裁剪**，用于稳定策略优化。该方法能够有效利用真实反馈，更好地使模型与用户偏好对齐，并显著提升 App Stay Time。

在快手/快手极速版上的大规模 A/B 测试表明，OneRec-V2 是有效的：它在平衡多目标推荐且不产生“跷跷板效应”的同时，使 App Stay Time 分别提升 0.467% 与 0.741%。该工作推动了生成式推荐在可扩展性以及真实世界反馈对齐方面的发展，是端到端推荐系统演进中的重要一步。

图 1 (原图略) 左：不同模型架构在 0.1B 至 8B 参数范围内的扩展曲线，其中 Lazy Decoder-only 模型展现出最优的扩展效率。右：在 1B 参数规模下，OneRec-V1 与 OneRec-V2 的对比。

1 引言

生成式人工智能已经在多个领域引发范式转变 (Achiam et al., 2023; Guo et al., 2025; Yang et al., 2025a)。尽管传统的级联式推荐架构经历了持续演化，但它们仍受到根本性瓶颈的限制：其固有的多阶段设计会导致计算资源割裂、优化目标不一致。生成式推荐则通过将推荐重新表述为端到端的序列生成问题，改变了这一范式 (Badrinath et al., 2025; Chen et al., 2024; Cui et al., 2022; Feng et al., 2022; Han et al., 2025; Kong et al., 2025; Rajput et al., 2023; Yang et al., 2025b; Zhai et al., 2024; Zhou et al., 2025)。这种统一的方法能够直接优化最终目标，获得较高的模型 FLOPs 利用率 (MFU)，并促进推荐系统社区与大型基础模型社区之间更紧密的融合。

尽管 OneRec-V1 (Zhou et al., 2025) 已经在工业部署中取得了相当大的成功，但其可扩展性与性能仍有进一步释放空间：

(1) **编码器-解码器架构中的计算分配低效**。OneRec-V1 采用编码器-解码器框架，其中用户历史交互序列先由编码器处理，再通过交叉注意力供解码器利用。虽然 OneRec-V1 的解码器参数量大于编码器，但其计算负载主要集中在编码器，因为编码器需要处理长用户交互序列，而解码器输入要短得多。正如第 2.1 节所示，在 OneRec-V1 的上下文长度为 512 时，上下文编码占总 FLOPs 的 97.66%，而解码器目标物品生成仅占 2.34%。这种严重失衡的资源分配带来了扩展性挑战，因为大部分计算预算都用于序列编码，而不是用于真正生成推荐决策的关键阶段。在相同计算预算下，这种失衡会限制模型向更大架构有效扩展的潜力。

(2) **仅依赖奖励模型的强化学习存在局限**。虽然 OneRec-V1 已经证明了基于奖励模型的强化学习在策略优化中的有效性，但该方法仍面临两个内在挑战。首先，采样效率有限，因为依赖奖励模型的方法需要额外计算资源进行在线生成和打分，因此只能对一小部分用户采样，以近似全局行为。其次，存在潜在的奖励黑客问题，即策略可能学会利用奖励模型中的某些模式或偏置，而这些模式并不会转化为真实收益。若能引入真实用户反馈以应对这些内在问题，就有望使策略更好地与用户偏好对齐，并获得更优结果。此外，OneRec 在大规模部署条件下，也为在持续反馈闭环中通过策略优化实现自我改进提供了关键机会。

在本文中，我们提出 OneRec-V2，通过惰性解码器架构和基于真实用户交互的偏好对齐来解决上述基础性限制。如图 2 所示，我们的主要贡献包括：

1. **惰性仅解码器架构**。我们提出一种精简的仅解码器架构，消除了传统编码器-解码器设计中的计算瓶颈。通过去除编码器模块并简化交叉注意力机制（移除 K/V 投影层），我们的惰性解码器在相同计算预算下实现了计算需求降低 94%、实际训练资源降低 90%，同时支持模型参数规模提升 $16\times$ （从 0.5B 扩展至 8B）。如图 1 所示，该架构不仅使 decoder-only Transformer 在工业级推荐系统中变得切实可用且高效，而且还表现出很强的扩展能力：在广泛模型尺度上，其收敛损失都紧密遵循 Hoffmann 等人 (2022) 提出的理论扩展律。这为未来大规模生成式推荐模型的发展提供了经验与理论上的双重指导。
2. **基于真实用户交互的偏好对齐**。我们提出一个完整的后训练框架，直接利用真实世界用户反馈信号，来应对生成式推荐系统中奖励建模的基础性挑战。具体包括：(i) **时长感知奖励塑形**，通过考虑视频时长差异，缓解原始观看时长信号中的固有偏差，使奖励更准确地反映内容质量而非时长本身；(ii) **自适应比率裁剪**，在保证策略优化收敛性的同时有效降低训练方差。实验显示，该方法显著提升了 App Stay Time。特别地，当引入 OneRec 自身推荐带来的流量分布模式后，我们观察到线上性能得到进一步放大，这表明模型优化与真实用户行为分布之间的对齐程度更高。

在拥有 4 亿日活用户的快手/快手极速版 App 上开展的大规模在线 A/B 测试表明，与 OneRec-V1 相比，OneRec-V2 取得了显著改进：App Stay Time 分别提升 0.467% 与 0.741%，同时能够有效平衡多种推荐目标而不出现“跷跷板效应”。

本文余下部分安排如下：首先，我们介绍 OneRec-V2 的架构以及预训练阶段的实证结果（第 2 节）；随后给出后训练方法（第 3 节）；接着通过在线 A/B 测试进行综合评估（第 4 节）；最后讨论当前局限，并提出未来可能的研究方向（第 5 节）。

图 2（原图略） OneRec-V2 的整体架构与后训练框架。左图展示了惰性仅解码器架构；右图展示了后训练阶段的偏好对齐流程。

2 惰性仅解码器架构

本节介绍基于惰性解码器的架构。第 2.1 节阐述 OneRec 模型架构的演化路径与设计思路；第 2.2 节给出 OneRec-V2 的惰性仅解码器架构，该架构在显著降低计算复杂度与内存消耗的同时，实现了更低的生成任务损失；最后，第 2.3 节给出全面的实证结果，用以验证惰性仅解码器设计的优越性，并分析生成式推荐系统中的扩展律。

2.1 设计原则

自回归模型已经成为现代自然语言处理中的主导范式，支撑了 GPT (Brown et al., 2020; Radford et al., 2019) 和 LLaMA (Touvron et al., 2023a,b) 等最先进的大语言模型。它们展现出卓越的可扩展性 (Hoffmann et al., 2022; Kaplan et al., 2020)，其成功来自一种**优雅而简洁的思想**：**采用统一架构，以自回归方式处理序列**。再结合大规模预训练能力 (Devlin et al., 2019; Raffel et al., 2020)，**基于 Transformer 的自回归模型**已经成为生成式人工智能应用的事实标准。

为了将这类架构适配到推荐系统中，第一步是构造适用于自回归训练的数据组织方式。**传统上，推荐系统的训练样本按照时间顺序组织为曝光序列**。然而，当这种组织方式与标准的 Next Token Prediction 目标结合时，会产生冗余，如图 3(a) 所示。避免冗余的一种做法是采用**以用户为中心的组织方式，即每个训练样本覆盖某个用户的完整交互历史**，如图 3(b) 所示。但这种方式会带来潜在的**时间泄漏风险** (Ji et al., 2023) 以及**流行度偏置问题**。已有大量研究 (Gangwar and Jain, 2021; Gharahighehi et al., 2021; Huang et al., 2022; Klimashevskaya et al.; Zhu et al., 2021) 试图缓解这些问题。

为解决上述问题，我们提出**仍按时间顺序组织数据，但仅对最新曝光物品施加训练损失**，如图 3(c) 所示，其中灰色物品不参与 next token prediction。由于历史曝光物品与最新曝光物品在模型中的作用不同，在此前的 OneRec-V1 (Zhou et al., 2025) 中，我们因此选择了编码器-解码器架构。

图 3（原图略）朴素曝光组织方式中，模式 $A \rightarrow B$ 会在多个曝光中被重复训练；以用户为中心的组织方式中，在时间 t_3 训练 User-2 的数据时，模型已经通过 User-1 在未来时刻 t_4 的交互学到了模式 $B \rightarrow C$ ；仅保留最新曝光的组织方式只在最新曝光上训练。

表 1: 与损失相关的目标解码所占计算比例 (以 1B 参数模型为例)

架构	指标	$N = 512$	$N = 3000$
Encoder-Decoder (0.5B:0.5B)	总计算量 (GFLOPs)	346	1988
	上下文编码 (GFLOPs)	338	1980
	目标解码 (GFLOPs)	8.1	8.1
	目标占比	2.34%	0.41%
Naive Decoder-Only (1B)	总计算量 (GFLOPs)	632	3618
	上下文编码 (GFLOPs)	614	3600
	目标解码 (GFLOPs)	18	18
	目标占比	2.85%	0.49%
Lazy Decoder-Only (1B)	总计算量 (GFLOPs)	18	18
	目标占比	$\approx 100\%$	$\approx 100\%$

如表 1 所示, 我们对计算细节进行了初步分析。计算过程可以分为两类: **上下文编码**与**目标解码**。

定义 1: 上下文编码。 指对用户上下文特征进行处理和变换的计算操作, 具体包括: (i) **编码器**中对上下文进行变换的操作; (ii) **解码器交叉注意力**中对上下文进行投影的操作。

定义 2: 目标解码。 指在**解码器**中对目标物品语义 token 进行处理和变换的计算操作, 具体包括: (i) 用于捕获语义 token 间依赖关系的自注意力; (ii) 执行非线性变换的前馈网络 (FFN); (iii) 交叉注意力中的查询与输出变换。

根据表 1, 在参数量相同的前提下, Encoder-Decoder 架构相较经典 Decoder-Only 架构几乎节省了一半计算量。然而, 这两类架构都仍然存在计算低效问题: **大部分计算都分配给了那些并不直接参与损失计算的 token**。对于典型的上下文长度 $N = 512$ (即 OneRec-V1 的设定), **实际参与损失计算的 FLOPs 占比不足 3%**; 随着上下文长度增加, 这一占比会进一步降低。更详细的计算分析见附录 B。为了将计算集中到目标物品的语义 token 上, 从而高效扩展到更大模型, 我们提出了 **惰性仅解码器架构**。

2.2 整体架构

本节介绍我们的新架构, 如图 4 所示。该架构通过两个关键创新, 从根本上重新思考了生成式推荐器的设计。

首先, 我们提出了一种**惰性仅解码器架构**, 不同于传统的编码器-解码器模型和朴素的仅解码器方案。我们的设计将上下文视为仅通过交叉注意力访问的静态条件信息, 从而在消除冗余计算的同时, 保留模型捕获复杂用户-物品交互的能力。

其次, 我们提出一种极其高效的惰性交叉注意力机制, 它不再包含键值投影。该设计结合分组查询注意力 (Grouped Query Attention, GQA) (Ainslie et al., 2023), 显著降低内存占用, 使得模型能够高效处理更长的用户历史。

图 4 (原图略) 所提出的惰性仅解码器生成式推荐器架构。Context Processor 将异构用户特征路径映射为统一的上下文表示, 并经过归一化后生成跨层共享的键值对, 供交叉注意力使用。Lazy Decoder 将 BOS token 与目标物品的语义 ID 输入多层 Transformer 块; 每个块包含: (1) 无键值投影的惰性交叉注意力, 可支持 GQA; (2) 因果自注意力; (3) 前馈网络。最终表示被投影为下一物品推荐所需的语义 ID。

2.2.1 上下文处理器 (Context Processor)

为了有效融合异构且多模态的用户行为信号, 我们设计了一个统一模块, 称为 Context Processor, 以便与下游基于注意力的解码器块无缝衔接。

具体而言, 用户画像与行为等异构输入会被拼接成统一序列, 即上下文 (context)。上下文中的每个元素都被处理到相同维度:

$$d_{\text{context}} = S_{kv} \cdot L_{kv} \cdot G_{kv} \cdot d_{\text{head}}, \quad (1)$$

其中, d_{head} 表示注意力头维度, G_{kv} 表示键值头分组数, S_{kv} 表示键值拆分系数, L_{kv} 表示键值层数。

随后, 上下文表示会被变换为供注意力机制使用的分层键值对。我们沿特征维度切分上下文张量, 以生成 L_{kv} 组键值对:

$$\text{Context} = [C_0, C_1, \dots, C_{S_{kv} \cdot L_{kv} - 1}], \quad (2)$$

其中 $C_{S_{kv} \cdot L_{kv} - 1} \in \mathbb{R}^{G_{kv} \cdot d_{\text{head}}}$ 。为简洁起见, 这里忽略序列维度。

对于每一层 $l \in \{0, 1, \dots, L_{kv} - 1\}$, 我们计算归一化后的键值对:

$$k_l = \text{RMSNorm}_{k,l}(C_{l \cdot S_{kv}}), \quad (3)$$

$$v_l = \begin{cases} \text{RMSNorm}_{v,l}(C_{l \cdot S_{kv} + 1}), & S_{kv} = 2 \quad (\text{键值分离}), \\ k_l, & S_{kv} = 1 \quad (\text{共享表示}). \end{cases} \quad (4)$$

因此, Context Processor 的最终输出为 $\{(k_0, v_0), \dots, (k_{L_{kv}-1}, v_{L_{kv}-1})\}$ 。

2.2.2 惰性解码器块 (Lazy Decoder Block)

Tokenizer。对于每个目标物品, 我们采用语义分词器生成 3 个语义 ID, 以刻画物品的多方面属性, 这与 OneRec-V1 (Zhou et al., 2025) 一致。在训练过程中, 我们使用前 2 个 ID, 并在序列前加上起始符 (beginning-of-sequence, BOS), 构成输入序列。随后, 这些 token 索引通过嵌入表映射为初始隐表示:

$$h^{(0)} = \text{Embed}([\text{BOS}, s_1, s_2]) \in \mathbb{R}^{3 \times d_{\text{model}}}. \quad (5)$$

块结构。 惰性解码器由 N_{layer} 个堆叠的 Transformer 块组成, 每个块包含三个主要组件: 交叉注意力、自注意力与前馈模块。对于第 l 层, 其变换过程为:

$$h_{\text{cross}}^{(l)} = h^{(l-1)} + \text{CrossAttn}(\text{RMSNorm}(h^{(l-1)}), k^{l_{kv}}, v^{l_{kv}}), \quad (6)$$

$$h_{\text{self}}^{(l)} = h_{\text{cross}}^{(l)} + \text{SelfAttn}(\text{RMSNorm}(h_{\text{cross}}^{(l)})), \quad (7)$$

$$h^{(l)} = h_{\text{self}}^{(l)} + \text{FFN}^{(l)}(\text{RMSNorm}(h_{\text{self}}^{(l)})), \quad (8)$$

其中, RMSNorm 表示均方根层归一化, 用于提升训练稳定性。

为在保持计算效率的同时提高模型容量, 我们采用混合架构: 在较深层中, 用 Mixture-of-Experts (MoE) 模块替换稠密前馈网络。遵循 DeepSeek-V3 (Liu et al., 2024), 我们使用一种无辅助损失的负载均衡策略, 以保证专家利用效率。

惰性交叉注意力: KV 共享。 为提高参数效率与计算效率, 多个惰性解码器块共享同一组由上下文处理器产生的键值对。对于当前层 l , 相应的键值索引为:

$$l_{kv} = \left\lfloor \frac{l \cdot L_{kv}}{N_{\text{layer}}} \right\rfloor. \quad (9)$$

该设计保证连续若干层共享同一上下文表示 $(k_{l_{kv}}, v_{l_{kv}})$, 其中 $k_{l_{kv}}, v_{l_{kv}} \in \mathbb{R}^{(N_s + T_{\text{short}} + T_{\text{long}}) \times G_{kv} \times d_{\text{head}}}$ 。

我们还通过统一键值表示进一步提升参数效率, 即令所有层均满足 $v_l = k_l$ 。这一做法利用了“键值投影绑定后仍可保持相近性能”的经验观察, 同时还能降低模型内存占用。

惰性交叉注意力: 分组查询注意力。 查询投影仍保持 $H_q = d_{\text{model}}/d_{\text{head}}$ 个注意力头, 而键值对仅使用 G_{kv} 个头分组, 通常有 $G_{kv} < H_q$ 。这一设计显著降低了上下文表示的内存占用以及注意力计算中的内存访问需求, 从而支持更长上下文和更大 batch size 下的高效扩展。

输出层。 最后一个解码器块输出的隐表示, 会经过位置相关的 RMSNorm 和线性层, 为每个语义 ID 生成预测。在训练时, 我们最大化目标物品语义 ID 序列 $[s_1, s_2, s_3]$ 的似然。

2.3 实证结果

为了验证惰性仅解码器架构的有效性, 我们从多个维度开展了系统的实证评估。我们不仅将该方法与经典架构进行比较, 还研究了关键架构创新的影响, 并探讨了稠密与稀疏模型变体的扩展性质。所有实验均在快手 2025 年 8 月 10 日至 14 日的曝光流式训练数据上完成, 使用相同采样比例与一致的全局 batch size。除非特别说明, 我们设定 $L_{kv} = 1$ 、 $S_{kv} = 1$ 、 $d_{\text{head}} = d_{\text{model}}/N_{\text{head}}$ 、 $G_{kv} = N_{\text{head}}$, 且 $(N_s + T_{\text{short}} + T_{\text{long}}) \approx 512$ 。在线部署时, 我们使用 1B 参数模型, 并将长时用户行为序列长度扩展到 $(N_s + T_{\text{short}} + T_{\text{long}}) \approx 3000$ 。

2.3.1 架构比较

图 5 (原图略) 三种模型尺度下不同架构的训练曲线。尽管能达到相近损失, Lazy Decoder-Only 架构所需 FLOPs 仅为经典架构的约十分之一。E1D1 与 E1D2 分别表示编码器与解码器参数比为 1:1 和 1:2。

表 2: 不同架构在不同模型尺度下的对比

架构	参数规模	GFLOPs	激活量	收敛损失
Enc:Dec=1:1	0.1B	25.64	4.21B	3.59
Enc:Dec=1:2	0.1B	17.72	2.92B	3.55
Naive Dec-Only	0.1B	63.78	7.52B	3.54
Lazy Dec-Only	0.1B	1.98	0.31B	3.57
Enc:Dec=1:1	0.5B	142.73	10.79B	3.35
Enc:Dec=1:2	0.5B	104.73	7.94B	3.32
Naive Dec-Only	0.5B	317.68	19.28B	*
Lazy Dec-Only	0.5B	9.55	0.77B	3.33
Enc:Dec=1:1	1B	296.36	17.63B	3.28
Enc:Dec=1:2	1B	204.21	12.20B	3.26
Naive Dec-Only	1B	634.83	31.53B	*
Lazy Dec-Only	1B	18.89	1.24B	3.27

我们比较了三类生成式推荐架构: 编码器-解码器架构 (OneRec-V1)、朴素仅解码器架构, 以及本文提出的惰性仅解码器架构。对于每个模型, 我们都在 3 个语义 token 上计算平均生成损失:

$$L_{\text{Gen}} = -\frac{1}{3} \sum_{i=1}^3 \log p(s_i \mid \text{BOS}, s_{<i}, \text{Context}), \quad (10)$$

其中, s_i 表示目标物品的第 i 个语义 ID, BOS 表示句首 token, context 是上下文处理器的输出, 包含用户静态特征与行为特征。该损失与 OneRec-V1 不同: 本文取 3 个 token 损失的平均值, 而 V1 使用的是求和形式。

表 2 与图 5 展示了不同模型尺度下的计算需求与收敛性能。尽管所需 FLOPs 和激活内存显著更低, 我们的惰性仅解码器架构仍能取得与传统方法相当的损失。

2.3.2 键值共享

我们的上下文处理器引入了两个关键参数, 用于灵活控制整体上下文维度: L_{kv} 与 S_{kv} 。参数 L_{kv} 决定跨层使用多少组不同的上下文表示, 即每 N_{layer}/L_{kv} 个连续解码器块共享同一组键值对; 参数 S_{kv} 则控制键和值是共享同一表示 ($S_{kv} = 1$), 还是使用独立投影 ($S_{kv} = 2$)。该设计在保持生成任务性能近似不变的同时, 降低了计算成本与激活内存。我们在一组参数、 $N_{\text{layer}} = 18$ 的稠密惰性解码器模型上进行了消融实验, 以研究这些设计选择的影响。

图 11(a) 表明, 即便采用较为激进的键值共享策略, 模型在整个训练过程中仍能保持有竞争力的损失, 这验证了我们高效上下文处理策略的合理性。

表 3: 键值共享策略对模型效率与性能的影响

L_{kv}	S_{kv}	GFLOPs	激活量	收敛损失
1	1	18.89	1.24B	3.27
1	2	19.19	1.33B	3.27
3	1	19.49	1.42B	3.27
9	1	21.27	1.99B	3.27
18	1	23.95	2.83B	3.27

2.3.3 分组查询注意力

表 4: 分组查询注意力对模型效率与性能的影响

G_{kv}	GFLOPs	激活量	KV 大小	收敛损失
14	18.89	1.24B	94M	3.27
7	18.74	1.19B	47M	3.28
2	18.64	1.16B	13M	3.28
1	18.62	1.15B	7M	3.27

分组查询注意力 (GQA) 在多个查询头之间共享键值头。在我们的惰性解码器架构中, 这一优化降低了交叉注意力中的激活内存与内存访问瓶颈, 从而在几乎不影响模型质量的情况下提高训练吞吐。我们在一一个 1B 参数、拥有 14 个注意力头的稠密惰性解码器模型上, 考察了不同键值头分组数 $G_{kv} \in \{1, 2, 7\}$ 的影响。

表 4 与图 11(b) 的结果表明, 使用不同分组数的 GQA 与完整注意力相比, 性能几乎一致, 但内存需求显著更低。

2.3.4 模型扩展

我们围绕惰性仅解码器架构开展了系统的扩展实验, 研究稠密配置与稀疏配置在不同模型尺度下的计算-性能权衡。

稠密模型扩展。 我们研究了从 0.1B 到 8B 参数范围内的稠密惰性解码器模型的扩展性质。表 5 给出了各个模型配置的结构超参数与收敛性能。

稀疏 Mixture-of-Experts。 为实现更高效的扩展, 我们进一步研究了一个 Mixture-of-Experts (MoE) 变体, 用稀疏专家路由替代稠密前馈网络。我们的 MoE 配置包含 53 个路由专家和 1 个共享专家, 总参数数量为 4B (其中每个 token 实际激活 0.5B)。模型对每个 token 采用 top-3 专家路由, MoE 中间层维度为 1408。该稀疏模型与 0.5B 稠密模型保持相同的基础架构, 只是在前两个惰性解码器块之后, 将前馈层替换为 MoE 层。

表 5: 模型扩展实验中的超参数配置与收敛损失

模型	参数规模	d_{model}	n_{layers}	n_{heads}	embed_dim	学习率	收敛损失
Dense	0.1B	640	12	10	32	5.00e-4	3.57
Dense	0.2B	896	12	14	45	3.54e-4	3.46
Dense	0.5B	1408	14	11	70	2.24e-4	3.33
Dense	1B	1792	18	14	90	1.58e-4	3.27
Dense	2B	2304	22	18	115	1.12e-4	3.23
Dense	4B	2944	26	23	147	7.91e-5	3.20
Dense	8B	3584	34	28	179	5.59e-5	3.19
MoE	4B (0.5B 激活)	1408	14	11	70	2.24e-4	3.22

图 6 (原图略) 稠密模型扩展律曲线。扩展律拟合形式为 $\hat{L}(N) \triangleq E + \frac{A}{N^\alpha}$, 其中 $E = 3.13$ 、 $A = 3660$ 、 $\alpha = 0.489$ 。

结果与分析。 图 7 展示了不同模型配置下的训练动态。实验揭示了惰性解码器架构在推荐系统中的若干扩展规律。我们还给出了不同模型尺度下, 随着训练预算增加, 损失如何下降的结果, 见图 12。

我们的经验结果与理论扩展律具有较好的一致性。一般形式的 Chinchilla 扩展律为

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

其中 N 是模型参数量, D 是训练 token 数 (Hoffmann et al., 2022)。而在我们的实验设定中, D 保持不变, 因此项 $\frac{B}{D^\beta}$ 是常数, 可以并入截距 E 。于是, 对于固定数据, 扩展律可化简为

$$\hat{L}(N) \triangleq E' + \frac{A}{N^\alpha}.$$

其中 $E' = E + \frac{B}{D^\beta}$ 。我们的经验结果与这一函数形式拟合得很好: $E = 3.13$ 、 $A = 3660$ 、 $\alpha = 0.489$, 如图 6 所示。

总参数量为 4B、每个 token 激活 0.5B 的 MoE 变体, 其收敛损失达到 3.22, 优于 2B 稠密模型, 同时其计算需求与 0.5B 稠密基线相当。与 0.5B 稠密模型相比, 该配置的损失降低了 0.11, 这表明稀疏架构对于推荐任务是有效的。

这些结果表明, 我们的惰性解码器架构能够有效扩展, 而 MoE 变体则为工业级推荐系统部署提供了特别有吸引力的权衡方案, 因为在这类场景中, 计算效率会直接影响服务成本与时延。

图 7 (原图略) 不同模型尺度下惰性解码器架构的训练动态。收敛损失从 0.1B 模型的 3.57 下降到 8B 模型的 3.19。图中的 4BA0.5B 表示 4B 总参数、0.5B 激活的 MoE 变体, 它在保持计算效率的同时取得了有竞争力的性能。

3 基于真实用户交互的偏好对齐

本节介绍 OneRec-V2 的后训练阶段。监督微调(Supervised Fine-Tuning)阶段与 OneRec-V1 相同，均使用流式曝光数据进行在线 L_{Gen} 损失训练，这也与预训练阶段所用损失保持一致。其主要目的在于捕捉用户的实时兴趣变化，同时避免模型相对预训练模型发生过大偏移。在 OneRec-V1 中，强化学习阶段仅基于奖励模型；而在 OneRec-V2 中，我们引入了以用户反馈信号作为奖励的强化学习。

3.1 基于用户反馈信号的强化学习

基于用户反馈定义奖励，可以避免奖励黑客问题，并且不需要额外的模型计算开销。然而，这一方法仍面临诸如多目标如何组合、正标签稀疏等挑战。在短视频推荐场景中，每条视频的播放时长是最密集的反馈信号，并且与最重要的线上指标（如 App Stay Time 和 LT7，即 7 天生命周期留存）高度相关。因此，我们设计了一种简单但有效的、基于播放时长的奖励。

图 8 (原图略) 时长感知奖励塑形示意图。用户观看历史中的视频按照时长分桶；对于某个目标视频，在其对应时长桶内计算其播放时长所处的分位数，作为该用户对该视频的偏好得分。

3.1.1 时长感知奖励塑形

尽管视频播放时长是衡量用户满意度的有效指标，但它天然受视频时长影响。为缓解这种偏差，我们提出了 **时长感知奖励塑形** 机制，如图 8 所示。该方法通过将目标视频的播放时长，与同一用户历史中“时长相近”的视频进行比较，对播放时长进行归一化。由于视频时长通常呈现长尾分布，我们采用对数分桶策略来划分历史视频。该策略会把时长映射到指数递增的区间中，从而得到更均衡、更有意义的同类比较集合。设映射函数 $F(d)$ 将时长为 d 的视频映射到离散桶索引 $b \in B$ ，则分桶函数定义为：

$$F(d) = \lfloor \log_{\beta}(d + \epsilon) \rfloor, \quad (11)$$

其中， β 是控制分桶粒度的对数底数， ϵ 是用于数值稳定性的极小常数（例如 10^{-6} ），以便处理极短视频。

设 $H_u = \{(d_k, p_k)\}_{k=1}^N$ 表示用户 u 的历史交互序列，其中 d_k 为视频时长， p_k 为观测到的播放时长。对于每个时长桶 b ，我们将播放时长的经验分布定义为：

$$P_{u,b} = \{p_j \mid (d_j, p_j) \in H_u, F(d_j) = b\}. \quad (12)$$

对于时长为 d_i 、播放时长为 p_i 的目标视频 i ，先确定其所属的桶 $b = F(d_i)$ 。随后，将其时长归一化后的参与度得分定义为 p_i 在用户历史分布 $P_{u,b}$ 中的经验分位数：

$$q_i = \frac{|\{p_j \in P_{u,b} \mid p_j \leq p_i\}|}{|P_{u,b}|}. \quad (13)$$

我们基于这一得分选择最有价值的样本作为正样本。在一个 batch 中，先将 q_i 按降序排序，再取其 25% 分位点（即上四分位）记为 τ_B 。对于具有显式负反馈的样本，例如“点踩”动作（ $neg_i = 1$ ），我们直接设 $A_i = -1$ ；其余样本都被过滤掉，这等价于令 $A_i = 0$ 。需要指出的是，我们没有再对 advantage 做归一化，因为这里对正负样本的定义已经足够严格；进一步归一化反而可能引入优化不一致，并导致性能下降。形式化定义如下：

$$A_i = \begin{cases} +1, & q_i > \tau_B \text{ 且 } neg_i = 0, \\ -1, & neg_i = 1, \\ 0, & \text{其他情况.} \end{cases} \quad (14)$$

这一策略能够有效筛选高质量正样本，同时引入直接的负反馈信号，从而得到更准确的用户偏好信号。

3.1.2 强化学习

Gradient-Bounded Policy Optimization. 近年来，大语言模型社区非常关注如何同时提升强化学习的有效性与稳定性。核心挑战在于：一方面需要增强探索以改进性能，另一方面又必须保持梯度稳定。为此，我们提出新的强化学习方法 **GBPO** (Gradient-Bounded Policy Optimization, 梯度有界策略优化)：

$$J_{\text{GBPO}}(\theta) = -\mathbb{E}_{u \sim P(U), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{\pi_{\theta}(o_i | u)}{\pi'_{\theta_{\text{old}}}(o_i | u)} \cdot A_i \right], \quad (15)$$

其中

$$\pi'_{\theta_{\text{old}}}(o_i | u) = \begin{cases} \max(\pi_{\theta_{\text{old}}}, \text{sg}(\pi_{\theta})), & A_i \geq 0, \\ \max(\pi_{\theta_{\text{old}}}, 1 - \text{sg}(\pi_{\theta})), & A_i < 0. \end{cases} \quad (16)$$

从该形式可以看出，GBPO 去除了对策略比率的显式裁剪操作，并为 $\pi_{\theta_{\text{old}}}$ 引入了一个动态边界。总体而言，GBPO 具有两个主要优点：

- **样本完全利用**：保留所有样本的梯度，鼓励模型进行更丰富的探索。
- **有界梯度稳定化**：用 BCE (二元交叉熵) 损失中更稳定的梯度为强化学习梯度设界，从而增强训练稳定性。

已有基于裁剪的方法。 在详细介绍 GBPO 之前，我们先简要回顾 LLM 场景中的一些现有强化学习方法。GRPO/PPO (Schulman et al., 2017; Shao et al., 2024) 通过裁剪操作丢弃策略比率过大或过小的样本，以防止训练过于激进。DAPO (Yu et al., 2025) 则通过放宽裁剪上界来减少样本约束，尤其是纳入更多低概率或高熵 token，从而提升探索多样性并改善强化学习性能。这些研究说明：适当放松裁剪约束、纳入更多样本，有助于增强探索并提升性能。

然而，这些方法并未对梯度稳定性进行完整而系统的考虑。尤其是对负样本而言，如果策略比率没有上界，就很容易导致梯度爆炸，进而使模型性能崩塌。Dual-clip (Ye et al., 2020) 对负样本的策略比率施加上界截断；虽然稳定性有所改善，但会丢弃过多负样本，从而减缓收敛。ECPO (Zhou et al., 2025) 则不是直接裁剪比率 $\pi_\theta/\pi_{\text{old}}$ ，而是对 π_{old} 进行裁剪，以缓解负样本上的梯度爆炸；这种做法在提升优化稳定性的同时，也保留了更多训练样本。类似地，CISPO (Chen et al., 2025) 与 GPPO (Su et al., 2025) 也采用了相关技术，使比率落在合理范围内，同时尽可能保留更多样本的梯度信息。在 OneRec-V1 中，我们采用的是 ECPO (Early Clipped GRPO)，其形式为：

$$J_{\text{ECPO}}(\theta) = -\mathbb{E}_{u \sim P(U), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(\frac{\pi_\theta(o_i | u)}{\pi'_{\theta_{\text{old}}}(o_i | u)} A_i, \text{clip} \left(\frac{\pi_\theta(o_i | u)}{\pi'_{\theta_{\text{old}}}(o_i | u)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) \right], \quad (17)$$

其中

$$\pi'_{\theta_{\text{old}}}(o_i | u) = \max \left(\frac{\text{sg}(\pi_\theta(o_i | u))}{1 + \epsilon + \delta}, \pi_{\theta_{\text{old}}}(o_i | u) \right), \quad \delta > 0. \quad (18)$$

梯度分析。 曝光样本既包括由 OneRec 生成的样本，也包括来自传统推荐流水线的样本。对于 OneRec 生成的曝光样本，我们使用曝光时的生成概率作为 π_{old} ；而对于传统流水线样本，由于该流水线过于复杂，无法获得其对应概率，因此我们将 π_{old} 近似为当前 OneRec 模型的生成概率，即 $\pi_{\text{old}} = \text{sg}(\pi_\theta)$ 。在这种情况下，策略比率恒为 1。传统强化学习方法通常认为“比率为 1”的样本训练是稳定的，因此不会对其进行截断；但实际上，如图 9 所示，这类样本仍可能由于负样本而引发梯度爆炸。

图 9 (原图略) GBPO 与传统基于比率裁剪方法的梯度对比。在训练负样本时，GBPO 的梯度显著更稳定。

从梯度角度看，对于这些样本中的某个特定 token i ，有：

$$J_{\text{ECPO}}^i(\theta) = -A_i \cdot \frac{\pi_\theta}{\text{sg}(\pi_\theta)}, \quad (19)$$

$$\frac{\partial J_{\text{ECPO}}^i(\theta)}{\partial \theta} = -A_i \cdot \frac{1}{\pi_\theta} \frac{\partial \pi_\theta}{\partial \theta}. \quad (20)$$

这说明当前 token 概率 π_θ 越小，梯度就越大。对于正样本来说，较小的概率意味着还有更大提升空间，因此更大的梯度是合理的；但对于负样本来说，较小的概率意味着本来就没有太多继续压低的空间，若梯度过大，就极易导致模型过拟合甚至崩塌。这一现象说明，传统裁剪方法无法完全解决强化学习梯度不稳定的问题，因为当比率等于 1 时，它们仍然无法避免梯度爆炸。而在 BCE 损失中，负样本同样会受到惩罚，但其梯度显著更稳定：

$$L_{\text{BCE}}(y, p_\theta) = -[y \log(p_\theta) + (1 - y) \log(1 - p_\theta)], \quad (21)$$

$$\frac{\partial L_{\text{BCE}}}{\partial \theta} = \begin{cases} -\frac{1}{p_\theta} \frac{\partial p_\theta}{\partial \theta}, & y = 1, \\ \frac{1}{1-p_\theta} \frac{\partial p_\theta}{\partial \theta}, & y = 0. \end{cases} \quad (22)$$

对于负样本，当前模型概率越小，其被继续压制时的梯度也越小，因此模型更稳定。基于这一观察，我们提出 GBPO：使用 BCE 中更稳定的梯度来为强化学习梯度设定边界。图 10 给出了其与传统方法的差异。

图 10 (原图略) GBPO 示意图。横轴为 $\pi_\theta/\pi_{\theta_{\text{old}}}$ ，纵轴为裁剪后的 $\pi_\theta/\pi_{\theta_{\text{old}}}$ 。“//”表示“无梯度”。相较于传统基于比率裁剪的方法，GBPO 的两点主要区别在于：其一，不会丢弃任何样本的梯度；其二，对于负样本，其比率的约束由与 π_θ 相关的动态边界决定。

3.1.3 实验

实验设置。 本节通过实验验证所定义用户反馈信号的有效性。为了快速验证，所有实验均在 0.5B 模型、上下文长度为 512 的设定下进行，基线模型为 OneRec-V1。在 OneRec-V1 的实验设定中，分配给实验组的线上流量只占总体的一小部分，因此训练样本几乎全部来自传统推荐流水线。在大语言模型领域，已有研究表明，使用模型自生成样本进行训练能够带来自我改进 (He et al., 2025)。随着 OneRec 目前已承担总流量的 25%，我们在当前场景中已经拥有足够数据来验证这一假设。为此，我们设计了两个实验组：

- **w/o OneRec Samples:** 仅使用传统推荐流水线产生的样本进行强化学习, 这与 OneRec-V1 的样本来源保持一致。
- **w/ OneRec Samples:** 引入由 OneRec 流水线产生的样本，其中也包括当前模型实验组自身生成的样本。换言之，该设定引入了 on-policy 强化学习。

如前所述，强化学习中的正样本定义为：按时长感知奖励得分排序后位于前 25% 分位的视频；负样本则由显式负反馈（如“点踩”）确定。需要注意的是，两组实验中的训练样本总量基本保持一致。强化学习损失采用 GBPO（式 (11)）。所有结果见表 6。

结果分析。 从表 6 可以得到如下观察。当仅使用传统流水线样本时，即与 OneRec-V1 的样本来源相同，引入基于用户反馈的强化学习后，App Stay Time、Watch Time 等时长相关指标显著提升，但部分其他指标（例如 Video View）有所下降。这说明，我们所设计的时长感知奖励与 App Stay Time 确实具有很强相关性。当进一步纳入 OneRec 流水线生成的样本后，几乎所有指标都显著改善，尤其是 Video View 从负增长转为正增长。这表明，基于用户反馈信号的强化学习能够实现自迭代优化，充分利用用户反馈来提升用户体验。

表 6: 基于用户反馈信号的强化学习在线 A/B 结果 (相对 OneRec-V1 的相对提升)

场景	指标	w/o OneRec Samples	w/ OneRec Samples
Kuaishou	App Stay Time	+0.165%	+0.227%
Kuaishou	Watch Time	+1.054%	+0.648%
Kuaishou	Video View	-0.901%	+0.716%
Kuaishou	Like	-0.186%	+2.897%
Kuaishou	Follow	+2.274%	+3.661%
Kuaishou	Comment	-4.982%	+6.392%
Kuaishou	Collect	-0.817%	+1.232%
Kuaishou	Forward	-2.162%	+3.426%
Kuaishou Lite	App Stay Time	+0.159%	+0.353%
Kuaishou Lite	Watch Time	+0.396%	+0.104%
Kuaishou Lite	Video View	-2.231%	+0.575%
Kuaishou Lite	Like	-0.534%	+4.956%
Kuaishou Lite	Follow	+1.809%	+4.800%
Kuaishou Lite	Comment	-4.860%	+5.067%
Kuaishou Lite	Collect	-0.377%	+2.701%
Kuaishou Lite	Forward	+0.775%	+5.783%

3.2 用户反馈信号与奖励模型

3.2.1 奖励模型的局限

本节比较 OneRec-V1 中依赖奖励模型的强化学习，与基于用户反馈信号的强化学习。虽然 OneRec-V1 通过大量实验验证了强化学习的有效性，但其性能受到采样概率有限的制约：由于资源限制，on-policy roll-out 只能在极小部分用户（1%）上进行。此外，奖励模型还容易受到奖励黑客影响。相比之下，用户反馈信号能够直接反映真实用户偏好，从而降低奖励黑客的风险。然而，在 OneRec 尚未全面部署之前，生成样本上的大规模真实用户反馈并不可得；随着 OneRec 完整部署，这类信号如今可以更有效地被利用，以实现更精确的自迭代优化。上一节中，我们已经验证了时长感知反馈信号的有效性；接下来，我们将进一步比较用户反馈与奖励模型的性能差异。

表 7: OneRec-V2 在不同强化学习训练方式下的在线 A/B 结果

场景	指标	Reward Model	User Feedback Signals	Hybrid
Kuaishou	App Stay Time	+0.269%	+0.299%	+0.283%
Kuaishou	Watch Time	+0.537%	+0.610%	+0.118%
Kuaishou	Video View	+0.505%	+0.647%	+0.647%
Kuaishou	Like	+6.552%	+2.435%	+7.010%
Kuaishou	Follow	+7.265%	+2.007%	+8.458%
Kuaishou	Comment	+15.472%	+0.944%	+8.763%
Kuaishou	Collect	+1.856%	+1.401%	+9.739%
Kuaishou	Forward	+12.024%	+0.803%	+5.270%
Kuaishou Lite	App Stay Time	+0.163%	+0.213%	+0.207%
Kuaishou Lite	Watch Time	+0.503%	+0.172%	-0.398%
Kuaishou Lite	Video View	+0.457%	+0.056%	+0.083%
Kuaishou Lite	Like	+7.798%	+4.008%	+6.267%
Kuaishou Lite	Follow	+12.242%	+4.421%	+11.705%
Kuaishou Lite	Comment	+11.284%	+3.958%	+7.002%
Kuaishou Lite	Collect	+4.468%	+1.731%	+3.495%
Kuaishou Lite	Forward	+15.919%	+7.704%	+6.670%

3.2.2 实验

实验设置。 我们设置了三组实验进行比较, 分别称为 Reward Model、User Feedback Signals 和 Hybrid。模型设定与第 3.1.3 节相同。评估指标也与前述实验一致, 既包括时长类指标, 也包括交互类指标。App Stay Time 是最重要的指标, 而其他指标可视作用户体验的参考值。表 7 所示结果都是相对于 OneRec-V1 的相对表现。

- **Reward Model:** 引入基于奖励模型的强化学习。与 OneRec-V1 的主要区别在于预训练生成模型的架构不同: OneRec-V1 使用编码器-解码器架构, 而 OneRec-V2 使用本文提出的 Lazy Decoder。
- **User Feedback Signals:** 引入基于用户反馈的强化学习, 并纳入模型自生成样本, 这与上一小节中的 “w/ OneRec Samples” 设定相同。
- **Hybrid:** 同时引入奖励模型和用户反馈信号, 两类样本彼此独立: 前者来自模型自身 rollout 采样, 后者则来自历史上已曝光给用户的样本。

结果分析。 从表 7 中, 我们可以总结出以下几点:

- 在奖励模型设定下, OneRec-V2 明显优于 OneRec-V1, 这进一步验证了 Lazy Decoder 架构带来的优势。
- 无论是基于奖励模型, 还是基于用户反馈, 强化学习都能同时带来时长指标与交互指标的双重收益。不过, 奖励模型更倾向于提升交互类指标, 而真实用户反馈更倾向于提升 App Stay Time。这是因为奖励模型输出的奖励是多种推荐目标的融合, 而我们根据用户反馈定义的奖励主要由视频播放时长计算得到。这也说明, 不同奖励定义会导致模型形成不同偏好, 这与 OneRec-V1 中的结论一致。

- 将二者结合时 (Hybrid)，尽管在时长类与交互类上的具体增益都不一定超过各自单独使用时的峰值，但性能损失很小，且 App Stay Time 与交互指标之间的平衡性更好。这是因为两种策略带来的收益存在部分重叠。虽然叠加后无法实现完全线性相加，但它们能够彼此互补。这也进一步说明了多样化奖励信号的重要性。未来，我们将进一步研究奖励信号的多样性与准确性问题。

表 8: OneRec-V2 相对 OneRec-V1 的在线 A/B 测试整体提升

场景	指标	OneRec-V2
Kuaishou	App Stay Time	+0.467%
Kuaishou	LT7	+0.069%
Kuaishou	Watch Time	+1.367%
Kuaishou	Video View	+0.331%
Kuaishou	Like	+3.924%
Kuaishou	Follow	+4.730%
Kuaishou	Comment	+5.394%
Kuaishou	Collect	+2.112%
Kuaishou	Forward	+3.183%
Kuaishou Lite	App Stay Time	+0.741%
Kuaishou Lite	LT7	+0.034%
Kuaishou Lite	Watch Time	+0.762%
Kuaishou Lite	Video View	+0.259%
Kuaishou Lite	Like	+5.393%
Kuaishou Lite	Follow	+5.627%
Kuaishou Lite	Comment	+5.013%
Kuaishou Lite	Collect	+3.202%
Kuaishou Lite	Forward	+7.958%

4 在线 A/B 测试

我们在快手的两个主要短视频场景中部署了 OneRec-V2：主站信息流和快手极速版信息流。这两个场景是平台流量最高的环境，总计服务 4 亿日活用户。评估采用 5% 流量实验组，观察周期为一周。所使用模型为 1B 参数版本，上下文长度为 3000，beam size 为 512。在线推理阶段，系统使用 L20 GPU，实现了 36ms 的时延以及 62% 的模型 FLOPs 利用率 (MFU)。为降低系统复杂度，该版本仅采用了 **User Feedback Signals**。我们的主要评估指标是 App Stay Time（衡量用户总停留时长）与 LT7（7 日用户生命周期留存）。

如表 8 所示，OneRec-V2 在两个平台上都取得了显著提升。此外，OneRec-V2 在点赞、关注、评论等所有用户交互类指标上也都有明显收益，这说明它能够引导多任务推荐系统朝着更平衡的方向演化，并有效缓解不同目标之间的“跷跷板效应”。

为进一步验证上述结论，我们还额外进行了一个关闭缓存的实验：在单独的 1% 实验组中，所有流量都直接请求 OneRec-V2（详细结果见附录 D）。这一更严格的评估进一步验证了用户参与度指标的显著提升，其中点赞、关注、评论、转发等交互指标在不同平台上的增益达到 9.6% 到 29.2%。不过，这些结果也揭示了生态层面的重要问题：冷启动视频曝光显著下降（快手与快手极速版分别下降 44.7% 与 36.7%），而 cluster density 则明显上升。

5 结论、局限与未来方向

本文在 OneRec-V1 的基础上提出了 OneRec-V2，并重点讨论了其扩展系统与奖励系统的设计。在扩展性方面，我们发现：尽管 OneRec-V1 利用 MoE 在解码器中配置了大量参数，但由于序列长度差异，上下文编码过程消耗了绝大部分计算资源，从而阻碍了进一步扩展和性能提升。因此，我们重新设计了模型架构，提出惰性仅解码器架构，将主要计算转移到解码阶段，从而支持进一步扩展（目前已扩展到 8B）。此外，我们还提出了一种能够有效利用真实用户反馈来对齐用户偏好的方法。与仅依赖奖励模型进行对齐的 V1 不同，V2 将真实用户反馈信号纳入训练，并通过创新设计建立了短期视频观看时长与长期满意度之间的关联。再结合 GBPO，我们实现了高度稳定的训练。严格的 A/B 实验已经证明了该框架的有效性。然而，我们的系统仍有进一步改进空间。例如：

1. **扩展性。**随着模型从 0.1B 扩展到 8B 参数，我们观察到损失持续下降，并且这一现象与 Hoffmann 等人（2022）提出的经验扩展律高度吻合。如图 6 所示，我们的结果与该扩展关系拟合得非常好。这验证了所选架构的有效性，也表明进一步扩展与架构创新仍有望带来更佳性能。
2. **奖励系统。**我们已经将真实用户反馈引入奖励系统，并证明了其有效性。但目前的方法仍然是通过规则将短期回报与长期回报联系起来，而不是让模型直接优化长期价值。未来，我们将朝这一方向继续优化，使模型能够围绕长期价值实现真正的自强化。

除在快手平台视频推荐场景中带来可观收益外，OneRec-V2 还已被部署到多个业务场景中，并产生了显著回报，例如（Wei et al., 2025）。我们相信，随着更多研究人员与工程师的持续迭代、验证与优化，这一系统还可以进一步提升。

A 贡献说明

在每个角色类别内部，作者均按照名字字母序排列。

核心贡献者。 Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, Pengfei Zheng, Qiang Luo, Qianqian Wang, Qigen Hu, Rongzhou Zhang, Rui Huang, Ruiming Tang, Shiyao Wang, Shujie Yang, Tao Wu, Wuchao Li, Xinchun Luo, Xingmei Wang, Yi Su, Yunfan Wu, Zexuan Cheng, Zhanyu Liu, Zixing Zhang.

其他贡献者。 Bin Zhang, Boxuan Wang, Chaoyi Ma, Chengru Song, Chenhui Wang, Chenglong Chu, Di Wang, Dongxue Meng, Dunju Zang, Fan Yang, Fangyu Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Honghui Bao, Hongyang Cao, Jiaming Huang, Jiapeng Chen, Jiaqiang Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Qiang Wang, Qidong Zhou, Shengzhe Wang, Shihui He, Shuang Yang, Siyang Mao, Sui Huang, Tiantian He, Tingting Gao, Wei Yuan, Xiao Liang, Xiaoxiao Xu, Xugang Liu, Yan Wang,

Yang Zhou, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yunfeng Zhao, Zhixin Ling, Ziming Li.

B 不同架构的计算复杂度

预备说明。 在实际推荐系统中，多个物品通常会同时曝光。一个关键优化是 **共享上下文压缩 (common context compression)**：当同一用户同时接收 k 个物品推荐时，共享的上下文信息（用户画像、历史行为）只需要处理一次，随后即可在所有目标物品之间复用。这样就把每个物品的有效上下文长度从 N 下降到约 N/k 。在快手场景中， $k = 5$ 。

Transformer 块中的主要计算组件 (Vaswani et al., 2017) 包括：(1) 前馈网络 (FFN)；(2) 注意力投影 (W_q, W_k, W_v, W_o)；(3) 注意力分数计算。它们的复杂度分别为：

$$\text{FFN: } O(L \cdot d_{\text{model}} \cdot d_{ff}) \approx O(L \cdot 4d_{\text{model}}^2), \quad (23)$$

$$\text{Attention Projections: } O(L \cdot 4d_{\text{model}}^2), \quad (24)$$

$$\text{Attention Scores: } O(L^2 \cdot d_{\text{model}}), \quad (25)$$

其中， L 是这些模块所处理的 token 数， d_{model} 是模型隐藏维度。需要注意的是，FFN 与注意力投影都可以近似写为 $O(L \cdot D)$ ，其中 D 表示相应模块的参数量。

编码器-解码器架构。 我们分析一个编码器与解码器各为 0.5B 参数的模型。在训练时，压缩后的上下文长度为 $N/5$ ，则其浮点运算量 (FLOPs) 可分解为：

$$\text{上下文变换 (Encoder) : } 6 \times 0.5B \times \frac{N}{5} = 0.6N \text{ GFLOPs}, \quad (26)$$

$$\text{上下文投影 (Cross-Attention) : } 6 \times 0.05B \times \frac{N}{5} = 0.06N \text{ GFLOPs}, \quad (27)$$

$$\text{上下文解码总量: } 0.6N + 0.06N = 0.66N \text{ GFLOPs}, \quad (28)$$

$$\text{目标解码: } 6 \times 0.45B \times 3 = 8.1 \text{ GFLOPs}, \quad (29)$$

$$\text{总计算量: } 0.66N + 8.1 \text{ GFLOPs}. \quad (30)$$

其中，系数 6 同时考虑了乘加操作（系数 2）以及前向-反向传播的比例（系数 3）。交叉注意力中的上下文投影矩阵 W_k, W_v 位于解码器内部，约占解码器参数的 10%（即 0.05B）。

这里忽略了注意力分数的计算。考虑具体模型配置：9 层编码器、9 层解码器，且 $d_{\text{model}} = 1792$ 。则注意力分数的计算量为：编码器部分 $6 \times 9 \times (N/5)^2 \times 1792 = 3.8N^2 \text{ KFLOPs}$ ，解码器部分 $6 \times 9 \times 3 \times N \times 1792 = 290N \text{ KFLOPs}$ 。当 $N = 512$ 时，这些量级相较 FFN 与注意力投影而言要小得多。

朴素仅解码器架构。 对于一个 $1B$ 参数的 decoder-only 模型，在因果注意力掩码下处理 $N/5 + 3$ 个 token 时，其计算量为：

$$\text{上下文解码: } 6 \times 1B \times \frac{N}{5} = 1.2N \text{ GFLOPs,} \quad (31)$$

$$\text{目标解码: } 6 \times 1B \times 3 = 18 \text{ GFLOPs,} \quad (32)$$

$$\text{总计算量: } 1.2N + 18 \text{ GFLOPs.} \quad (33)$$

C 补充实证结果

我们进行了额外实验，用于研究 OneRec-V2 模型中模型规模、计算预算与训练损失之间的关系。图 12 给出了不同规模模型在总计算量（以 FLOPs 衡量）作为横轴时的平滑生成训练损失曲线。总体而言，更大的模型需要更多计算资源才能达到相同损失水平，但它们也会收敛到更低的损失点，这与大语言模型领域的普遍观察一致。

图 11（原图略）不同交叉注意力配置下的训练损失曲线。左图表示键值共享策略，右图表示分组查询注意力策略。两种策略对收敛损失影响都很小，但都显著提升了计算效率。

图 12（原图略）OneRec-V2 不同模型规模下，生成训练损失随总计算预算（FLOPs）变化的平滑曲线，展示了其扩展行为与收敛模式。橙色曲线连接了各模型达到的最小损失点。

D 关闭缓存时的在线性能

如第 4 节所述，正文实验中的流量设置为：实验组占总流量的 5%，且在该实验组中，仅有 25% 的降级流量应用 OneRec-V2。为了进行更严格的比较，我们额外划分了一个关闭缓存的 1% 实验组；在该组中，所有流量都直接请求 OneRec-V2。性能结果见表 9。

当所有流量都请求 OneRec-V2 时，我们观察到关键参与度指标显著提升，其中包括观看时长以及多项用户交互指标。具体而言，点赞、关注、评论和转发等交互类指标在不同平台上的提升幅度达到 9.6% 至 29.2%。然而，一些生态层面的指标却表现出值得关注的趋势。尤其是，冷启动视频曝光量出现明显下降（快手下降 44.7%，快手极速版下降 36.7%），同时 cluster density 也显著上升（分别为 11.7% 和 7.9%）。这构成了一个关键挑战，值得在未来工作中认真对待。

表 9: 关闭缓存、所有流量均请求 OneRec-V2 时，OneRec-V2 相对 OneRec-V1 的提升情况

场景	指标	OneRec-V2
Kuaishou	App Stay Time	+0.405%
Kuaishou	Watch Time	+0.513%
Kuaishou	Video View	+0.938%
Kuaishou	Like	+15.024%
Kuaishou	Follow	+15.755%
Kuaishou	Comment	+29.249%
Kuaishou	Collect	+9.640%
Kuaishou	Forward	+24.741%
Kuaishou	Cold-Start Video View	-44.704%
Kuaishou	Cluster Density	+11.692%
Kuaishou Lite	App Stay Time	+0.958%
Kuaishou Lite	Watch Time	+2.456%
Kuaishou Lite	Video View	-1.121%
Kuaishou Lite	Like	+12.783%
Kuaishou Lite	Follow	+21.376%
Kuaishou Lite	Comment	+16.975%
Kuaishou Lite	Collect	+12.886%
Kuaishou Lite	Forward	+30.957%
Kuaishou Lite	Cold-Start Video View	-36.730%
Kuaishou Lite	Cluster Density	+7.933%

参考文献说明

本文参考文献条目未逐条翻译，建议在阅读本中文译文时与原始英文论文 PDF 对照使用。